

JAVASCRIPT FUNDAMENTALS

Js



Introduction to JavaScript

JavaScript is the most important language in ethical hacking today because:

- Almost every web application attack happens in the browser (XSS, CSRF, DOM manipulation, client-side logic bypass)
- Most modern web apps use JavaScript frameworks (React, Vue, Angular, Next.js, etc.)
- You can write powerful security tools in Node.js (recon, scanners, exploit frameworks)
- Understanding JS deeply helps you exploit, bypass, and secure client-side code

History:

- 1995: Created by Brendan Eich in just 10 days at Netscape → originally called Mocha → then LiveScript → finally JavaScript (marketing trick to ride Java's popularity)
- 1996–1999: Standardized as ECMAScript (ES1 → ES3)
- 2009: ECMAScript 5 (strict mode, JSON support)
- 2015: ES6 / ES2015 → huge modern upgrade (let/const, arrow functions, classes, promises, modules...)
- 2016–2025: Yearly releases (ES2016 → ES2025) → now very modern & powerful

What is JavaScript?

High-level, interpreted, dynamic, multi-paradigm scripting language primarily running in browsers, but also on servers (Node.js), mobile (React Native), desktop (Electron), IoT, etc.

Key Characteristics

- Interpreted (not compiled)
- Dynamically typed (no need to declare types)
- Prototype-based (not class-based originally)
- First-class functions (functions are values)
- Event-driven & asynchronous (callbacks, promises, async/await)

Advantages of JavaScript (Especially for Ethical Hacking)

Advantage	Why it matters for hacking / pentesting
Runs everywhere	Browser, server (Node.js), mobile, desktop → write once, exploit everywhere
Huge ecosystem	npm → 2.5M+ packages → many security tools (axios, cheerio, puppeteer...)
Full control over DOM	Perfect for XSS payloads, DOM XSS, client-side logic bypass
Asynchronous by nature	Great for brute-force, rate-limit bypass, WebSocket attacks
Modern & fast	V8 engine (Chrome, Node) → extremely fast execution
Huge community & job market	Most in-demand language → good for career too
Can be used offensively & defensively	Write exploits, scanners, and secure code alike

Applications of JavaScript in Ethical Hacking

- **Client-side attacks** → XSS, DOM-based XSS, Open Redirects, Client-side logic bypass
- **Web scraping/recon** → Puppeteer, Playwright, Cheerio
- **Automation** → Burp Suite extensions, custom scanners, Nuclei templates
- **Exploit development** → Browser exploits, prototype pollution, deserialization
- **Post-exploitation** → BeEF (Browser Exploitation Framework), keyloggers
- **Node.js tools** → subdomain brute-forcer, port scanner, password cracker

Variables, Data Types & Operators

Variables:

```
var name = "Raju";           // function-scoped, hoisted
```

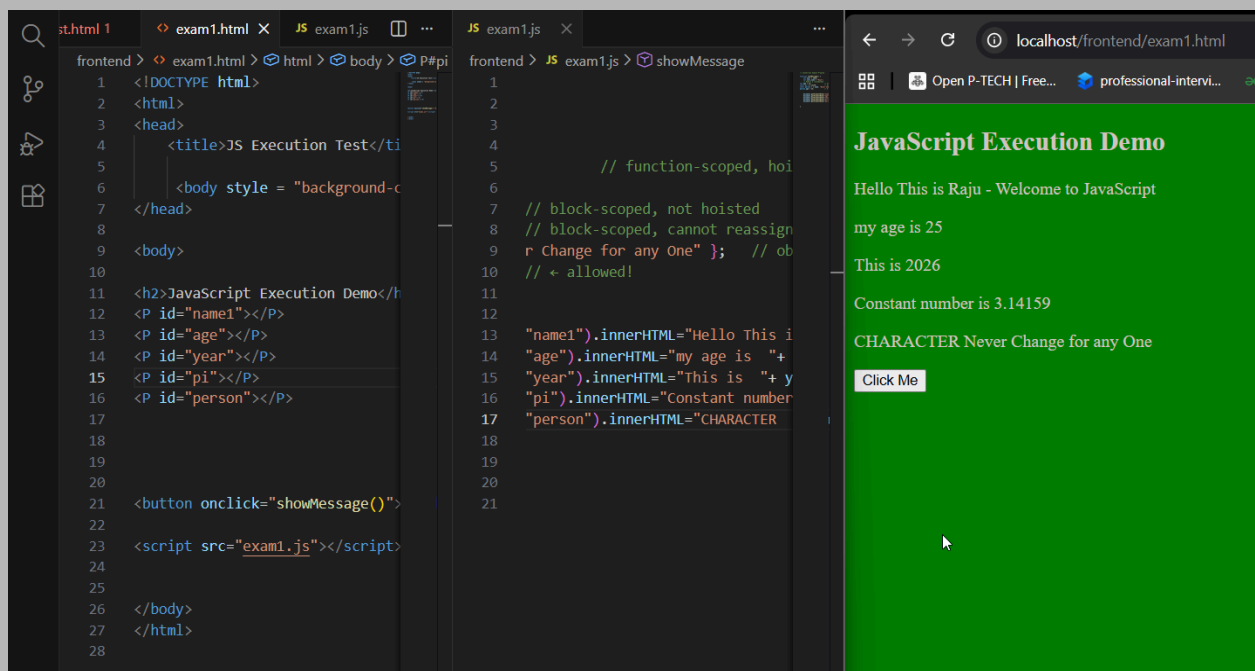
// Modern & recommended

```
let age = 25;                 // block-scoped, not hoisted
```

```
const pi = 3.14159;          // block-scoped, cannot reassign
```

```
const person = { name: "Raju" }; // object reference can change
```

```
person.age = 25;              // ← allowed!
```



Data Types (Dynamic – no declaration needed)

Type	Example	Notes / Hacking relevance
Number	<code>42, 3.14, NaN, Infinity</code>	Type juggling → <code>==</code> vs <code>===</code> attacks
String	<code>"hello", 'world', `template`</code>	Template literals → great for payload injection
Boolean	<code>true, false</code>	
Undefined	<code>let x;</code>	Common in prototype pollution
Null	<code>null</code>	
Object	<code>{ name: "Raju" }</code> , arrays, functions	Most important – everything is object under the hood
Symbol	<code>Symbol('id')</code>	Unique keys (ES6+)
BigInt	<code>123n, 9007199254740991n</code>	For very large numbers (crypto relevant)

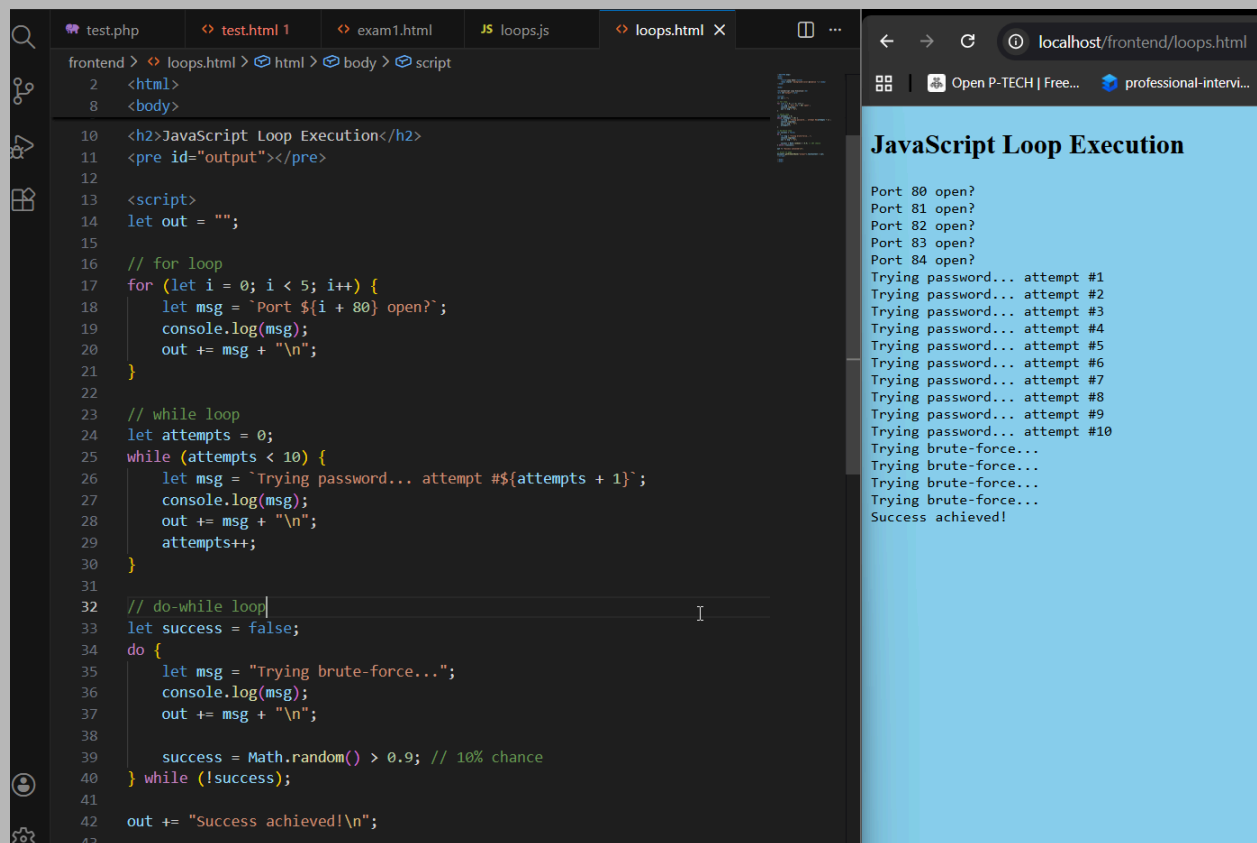
Operators

- **Arithmetic:** +, -, *, /, %, ** (power)
- **Comparison:** == (loose), === (strict), !=, !==, >, <
- **Logical:** &&, ||, !
- **Type coercion dangers:**

```
console.log("2" == 2);    // true (loose equality)
console.log("2" === 2);   // false (strict – use this!)
console.log([] == ![]);   // true – famous JS WTF
```

Loops

- **for** (classic)
- **while**
- **do-while** (run at least once)
- **for...of** (best for arrays)
- **for...in** (for objects – keys)



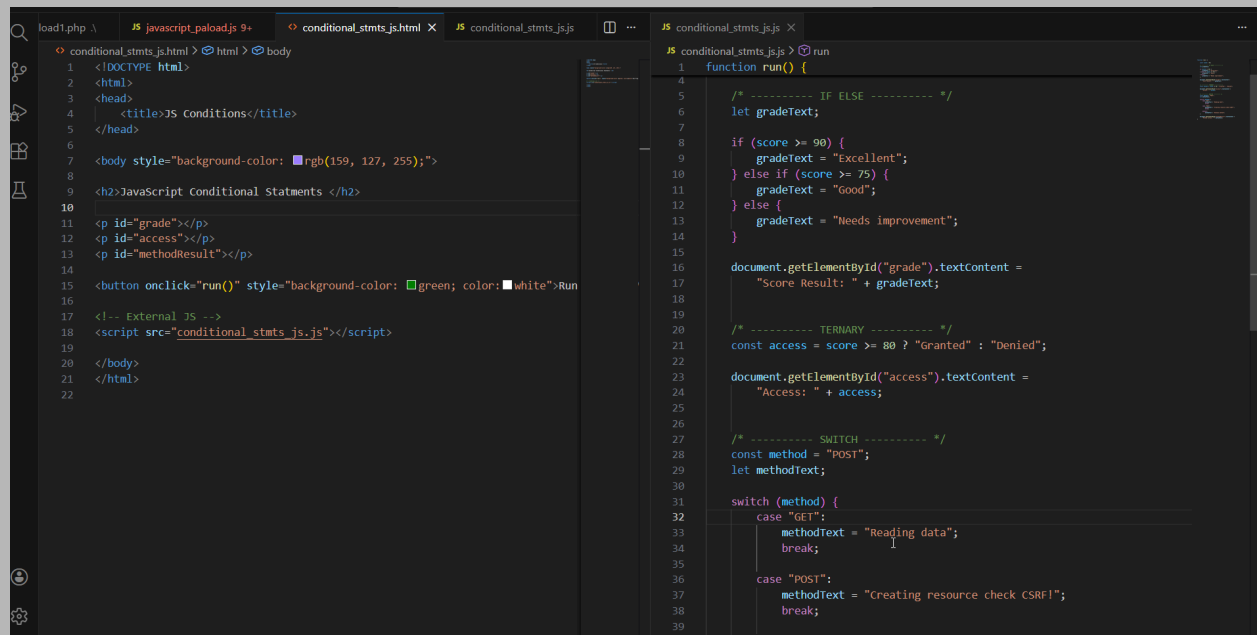
```
test.php test.html 1 exam1.html JS loops.js loops.html X
frontend > loops.html > html > body > script
2 <html>
8 </body>
10 <h2>JavaScript Loop Execution</h2>
11 <pre id="output"></pre>
12
13 <script>
14 let out = "";
15
16 // for loop
17 for (let i = 0; i < 5; i++) {
18   let msg = `Port ${i + 80} open?`;
19   console.log(msg);
20   out += msg + "\n";
21 }
22
23 // while loop
24 let attempts = 0;
25 while (attempts < 10) {
26   let msg = `Trying password... attempt #${attempts + 1}`;
27   console.log(msg);
28   out += msg + "\n";
29   attempts++;
30 }
31
32 // do-while loop
33 let success = false;
34 do {
35   let msg = "Trying brute-force...";
36   console.log(msg);
37   out += msg + "\n";
38
39   success = Math.random() > 0.9; // 10% chance
40 } while (!success);
41
42 out += "Success achieved!\n";
43
```

JavaScript Loop Execution

Port 80 open?
Port 81 open?
Port 82 open?
Port 83 open?
Port 84 open?
Trying password... attempt #1
Trying password... attempt #2
Trying password... attempt #3
Trying password... attempt #4
Trying password... attempt #5
Trying password... attempt #6
Trying password... attempt #7
Trying password... attempt #8
Trying password... attempt #9
Trying password... attempt #10
Trying brute-force...
Trying brute-force...
Trying brute-force...
Trying brute-force...
Success achieved!

6. Conditional Statements

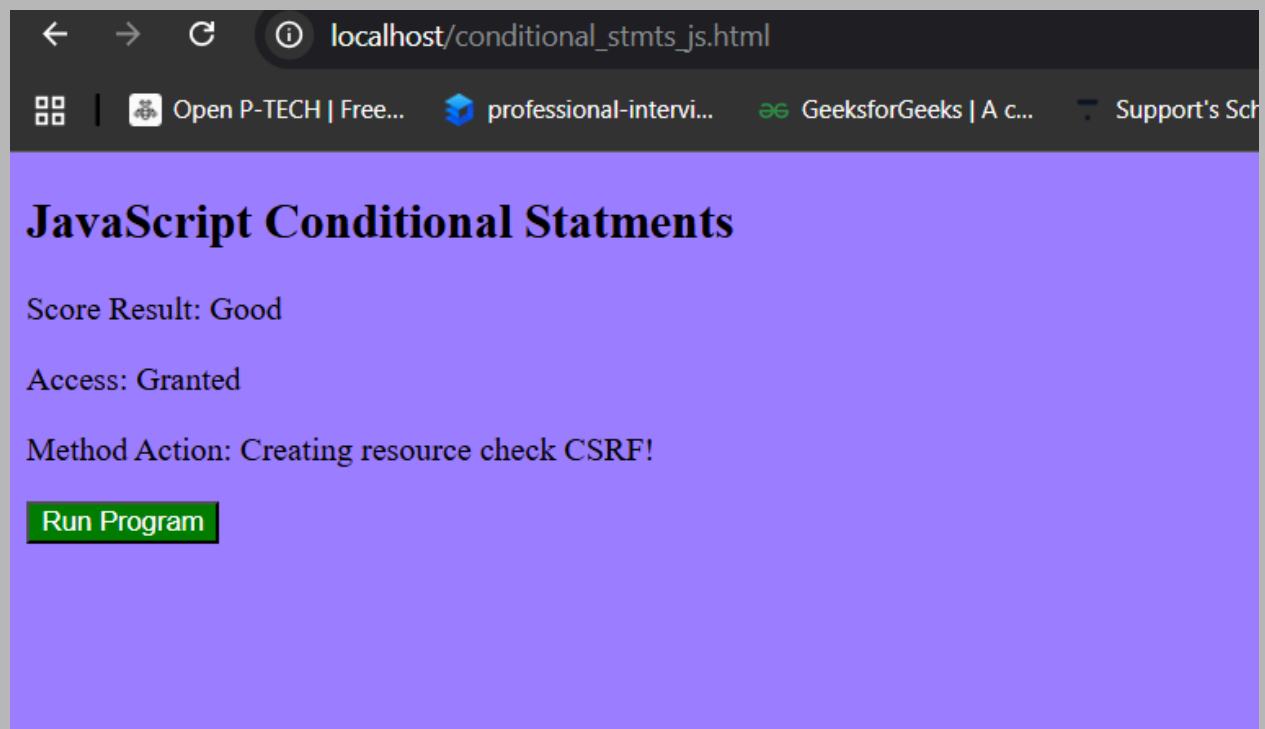
Ternary operator (short if-else), Switch



```
1 </DOCTYPE html>
2 <html>
3 <head>
4 <title>JS Conditions</title>
5 </head>
6
7 <body style="background-color: rgb(159, 127, 255);">
8
9 <h2>JavaScript Conditional Statements </h2>
10
11 <p id="grade"></p>
12 <p id="access"></p>
13 <p id="methodResult"></p>
14
15 <button onclick="run()" style="background-color: green; color: white">Run
16
17 <!-- External JS -->
18 <script src="conditional_stmts.js"></script>
19
20 </body>
21 </html>
22
```

```
1 function run() {
2
3 /* ----- IF ELSE ----- */
4
5 let gradeText;
6
7
8 if (score >= 90) {
9     gradeText = "Excellent";
10 } else if (score >= 75) {
11     gradeText = "Good";
12 } else {
13     gradeText = "Needs improvement";
14 }
15
16 document.getElementById("grade").textContent =
17     "Score Result: " + gradeText;
18
19
20 /* ----- TERNARY ----- */
21 const access = score >= 80 ? "Granted" : "Denied";
22
23 document.getElementById("access").textContent =
24     "Access: " + access;
25
26
27 /* ----- SWITCH ----- */
28 const method = "POST";
29 let methodText;
30
31 switch (method) {
32     case "GET":
33         methodText = "Reading data";
34         break;
35
36     case "POST":
37         methodText = "Creating resource check CSRF!";
38         break;
39 }
```

Output:



Functions in JavaScript

Types of Functions

Declaration: function add(a,b){return a+b} (hoisted)

Expression: const sub = function(a,b){return a-b}

Arrow: const mul = (a,b) => a*b (no own this, concise)

1. **Function Declaration** (hoisted)
2. **Function Expression** (not hoisted)
3. **Arrow Function** (ES6 – most common today)
4. **Immediately Invoked Function Expression (IIFE)**,
5. **Function Parameters** (Modern)

```
JS javascript_paload.js 9+ conditional_stmts_js.html functions_js.html X JS functions_js.js ... JS functions_js.js X
functions_js.html > html > body
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>JavaScript Functions </title>
5 </head>
6
7 <body style="background-color: rgb(145, 110, 110); color: white">
8
9 <h2>JavaScript Function Types</h2>
10
11 <button onclick="runFunctions()">Functions</button>
12
13 <pre id="output"></pre>
14
15 <!-- External JS -->
16 <script src="functions_js.js"></script>
17
18 </body>
19 </html>
20

JS functions_js.js > runFunctions
1 function runFunctions() {
17
18
19 /* ----- Arrow Functions ----- */
20 const multiply = (a, b) => a * b;
21 const square = x => x ** 2;
22 const sayHi = () => "Hi!";
23
24 out += "Multiply: " + multiply(4, 6) + "\n";
25 out += "Square: " + square(5) + "\n";
26 out += sayHi() + "\n";
27
28
29 /* ----- IIFE ----- */
30 (function() {
31   out += "IIFE ran immediately in private scope used\n";
32 })();
33
34
35 /* ----- Modern Parameters ----- */
36 function scan(target, port = 80, timeout = 5000, ...extraOptions) {
37   out += `Scanning ${target}:${port} timeout=${timeout}ms\n`;
38   out += "Extra options: " + extraOptions.join(", ") + "\n";
39 }
40
41 scan("example.com");
42 scan("example.com", 443, 3000, "--fast", "-sv");
43
44
45 /* ----- Print to Page ----- */
46 document.getElementById("output").textContent = out;
47
48 }
49
```


Output:



Arrays & Multidimensional Arrays

- 1D Array
- Multidimensional (array of objects – most common)

```
arrays.js.html > html > body
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Arrays</title>
5 </head>
6
7 <body style="background-color:rgb(38, 5, 38); color:yellowgreen">
8
9 <h2>JavaScript Arrays & Multidimensional Arrays</h2>
10
11 <button onclick="runArray()" style="background-color:greenyellow">Array</button>
12
13 <pre id="output"></pre>
14
15 <!-- External JS -->
16 <script src="arrays.js"></script>
17
18 </body>
19 </html>
20

JS arrays.js
1 function runArray() {
2
3   /* ----- 1D Array ----- */
4   const ips = ["192.168.1.1", "10.0.0.5"];
5
6   out += "IP List:\n";
7   ips.forEach(ip => {
8     out += ip + "\n";
9   });
10
11   /* ----- Multidimensional (Array of Objects) ----- */
12   const scanResults = [
13     { ip: "192.168.1.1", port: 80, status: "open", service: "http" },
14     { ip: "10.0.0.5", port: 22, status: "open", service: "ssh" }
15   ];
16
17   /* ----- Access Specific Value ----- */
18   out += "\nSecond service: " + scanResults[1].service + "\n";
19
20   /* ----- Loop Through ----- */
21   out += "\nOpen Services:\n";
22
23   scanResults.forEach(result => {
24     if (result.status === "open") {
25       out += `Vulnerable service? → ${result.service} (${result.ip}:${result.port})\n`;
26     }
27   });
28
29   /* ----- Print to Page ----- */
30   document.getElementById("output").textContent = out;
31   console.log(scanResults[1].service);
32 }
```

output:

```
localhost/arrays_js.html
Open P-TECH | Free... professional-intervi... GeeksforGeeks | A c...

JavaScript Arrays & Multidimensional Arrays

Array

IP List:
192.168.1.1
10.0.0.5

Second service: ssh

Open Services:
Vulnerable service? → http (192.168.1.1:80)
Vulnerable service? → ssh (10.0.0.5:22)
```

Category	Most Important Ones	Hacking Relevance
Array	<code>push</code> , <code>pop</code> , <code>shift</code> , <code>unshift</code> , <code>splice</code> , <code>slice</code> , <code>map</code> , <code>filter</code> , <code>reduce</code> , <code>forEach</code> , <code>find</code> , <code>some</code> , <code>every</code>	Data processing, payload crafting
String	<code>includes</code> , <code>startsWith</code> , <code>endsWith</code> , <code>trim</code> , <code>replace</code> , <code>split</code> , <code>match</code>	Payload detection, sanitization
Object	<code>Object.keys()</code> , <code>Object.values()</code> , <code>Object.entries()</code> , <code>Object.assign()</code> , <code>Object.freeze()</code>	Prototype pollution, object manipulation
JSON	<code>JSON.parse()</code> , <code>JSON.stringify()</code>	Parsing API responses / crafting payloads
Console	<code>console.log</code> , <code>console.table</code> , <code>console.error</code>	Debugging exploits
DOM	<code>document.querySelector</code> , <code>addEventListener</code>	XSS, DOM manipulation
Fetch / Async	<code>fetch()</code> , <code>async/await</code>	API abuse, CORS bypass testing

Example: Simple Login Form Validation (Client-side)

Using HTML, CSS and JavaScript to create this Login Form

```
login.js.html > html > head > link
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>JS Login Validator</title>
6
7   <!-- External CSS -->
8   <link rel="stylesheet" href="login_js.css">
9 </head>
10
11 <body>
12
13 <div class="card">
14   <!-- Secure Login (Client-side check) -->
15   <h2>Login</h2>
16
17   <input type="email" id="email" placeholder="Email">
18   <input type="password" id="password" placeholder="Password">
19
20   <button onclick="validateLogin()">Login</button>
21
22   <p id="message" class="error"></p>
23 </div>
24
25 <!-- External JavaScript -->
26 <script src="login_js.js"></script>
27
28 </body>
29 </html>
30

# login_js.css > .card
1 body {
2   font-family: system-ui;
3   max-width: 500px;
4   margin: 3rem auto;
5   padding: 1rem;
6 }
7
8 .card {
9   background: #f8f9fa;
10  padding: 2rem;
11  border-radius: 12px;
12  text-align: center;
13  box-shadow: 0 4px 15px #c6c8ca;
14 }
15
16 input, button {
17   display: block;
18   width: 100%;
19   padding: 0.9rem;
20   margin: 0.8rem 0;
21   border-radius: 6px;
22   border: 1px solid #c6c8ca;
23 }
24
25 button {
26   background: #007bff;
27   color: white;
28   border: none;
29   cursor: pointer;
30 }
31
32 button:hover {
33   opacity: 0.9;
34 }
35
36 .error {
37   color: #dc3545;

login.js.js > validateLogin
1 function validateLogin(email = "", password = "") {
2
3   // Get values if not passed
4   if (!email) email = document.getElementById("email").value.trim;
5   if (!password) password = document.getElementById("password").value.trim;
6
7   // Email check
8   if (!email.includes("@")) {
9     msg.textContent = "Invalid email format";
10    return false;
11  }
12
13  // Length check
14  if (password.length < 8) {
15    msg.textContent = "Password must be at least 8 characters";
16    return false;
17  }
18
19  // Strength check
20  if (!/[A-Z]/.test(password) || !/[0-9]/.test(password)) {
21    msg.textContent = "Password must contain uppercase & number";
22    return false;
23  }
24
25  // Success
26  msg.style.color = "#16a34a";
27  msg.textContent = "Login looks good! (Real verification happens server-side)";
28  msg.textContent += "Loggedin Successfully!";
29
30  return true;
31 }
```

Output:

